

Chapter 8

Kubernetes in Action 1st Edition

Pod 메타데이터와 API Server 접근

Accessing pod metadata and other resources from applications



신재근 (Jagger)

8장의 핵심 질문

Pod 이름, IP, Node처럼 배포 전에는 알 수 없는 값을
앱에서 어떻게 알 수 있는가?

세 가지 접근법

방법	범위	용도
Downward API	Pod 자신	이름, IP, 라벨, 리소스
API Server 직접	클러스터 전체	다른 리소스 조회
Ambassador	클러스터 전체	인증 복잡성 제거

6~8장 흐름

6장: 데이터를 어디에 저장? → Volume → PVC → StorageClass
7장: 설정을 어떻게 주입? → ConfigMap/Secret → env/Volume
8장: 런타임 정보를 조회? → Downward API + API Server

7장은 "밖에서 넣어주는" 설정

8장은 "안에서 스스로 알아내는" 정보

Part 1: Downward API란?

REST 엔드포인트가 아니다!

Pod 명세(spec) / 상태(status)에서 값을 추출하여
컨테이너에 내려보내는(downward) 메커니즘

```
Pod manifest (metadata, spec, status)
      |
      | Downward API
      ▼
Container
env: POD_NAME=downward
/etc/downward/labels
```

노출 가능한 메타데이터

정보	env	Volume	필드
Pod 이름	✓	✓	metadata.name
Namespace	✓	✓	metadata.namespace
Pod IP	✓	✓	status.podIP
Node 이름	✓	✓	spec.nodeName
CPU/Memory	✓	✓	resourceFieldRef
Labels	✗	✓	metadata.labels
Annotations	✗	✓	metadata.annotations

Labels/Annotations → Volume만 가능 (이유: 뒤에서 설명)

환경변수 방식 — fieldRef

```
env:  
- name: POD_NAME  
  valueFrom:  
    fieldRef:  
      fieldPath: metadata.name  
- name: POD_IP  
  valueFrom:  
    fieldRef:  
      fieldPath: status.podIP
```

Pod의 `metadata.name` 을 환경변수 `POD_NAME` 에 주입

환경변수 방식 — resourceFieldRef

```
env:  
- name: CONTAINER_CPU_REQUEST_MILLICORES  
  valueFrom:  
    resourceFieldRef:  
      resource: requests.cpu  
      divisor: 1m          # CPU 15m ÷ 1m = 15  
- name: CONTAINER_MEMORY_LIMIT_KIBIBYTES  
  valueFrom:  
    resourceFieldRef:  
      resource: limits.memory  
      divisor: 1Ki         # 4Mi ÷ 1Ki = 4096
```

divisor: 원하는 단위로 나누어 정수값 제공

데모: Downward API env

```
$ kubectl apply -f downward-api-env.yaml

$ kubectl exec downward -- env | grep -E "POD_|NODE_|CONTAINER_"
POD_NAME=downward
POD_NAMESPACE=default
POD_IP=172.21.59.141
NODE_NAME=minikube-m04
CONTAINER_CPU_REQUEST_MILLICORES=15
CONTAINER_MEMORY_LIMIT_KIBIBYTES=4096
```

배포 전에 알 수 없던 Pod IP, Node 이름이 정확히 주입됨

Volume 방식 — downwardAPI 볼륨

```
volumes:  
- name: downward  
  downwardAPI:  
    items:  
    - path: "labels"  
      fieldRef:  
        fieldPath: metadata.labels  
    - path: "containerCpuRequestMillicores"  
      resourceFieldRef:  
        containerName: main      # Volume은 Pod 레벨 → 컨테이너 명시 필요  
        resource: requests.cpu  
        divisor: 1m
```

데모: Downward API Volume

```
$ kubectl exec downward-volume -- ls /etc/downward
annotations  containerCpuRequestMilliCores
containerMemoryLimitBytes  labels  podName  podNamespace

$ kubectl exec downward-volume -- cat /etc/downward/labels
app="downward-demo"
version="1.0"

$ kubectl exec downward-volume -- cat /etc/downward/containerCpuRequestMilliCores
15
```



데모: 라벨 동적 갱신

```
# Before
$ cat /etc/downward/labels
app="downward-demo"
version="1.0"

# 라벨 추가
$ kubectl label pod downward-volume env=test

# After (자동 갱신!)
$ cat /etc/downward/labels
app="downward-demo"
env="test"           ← 새로 추가됨!
version="1.0"
```

환경변수는 한번 설정되면 변경 불가 → Labels/Annotations는 Volume만 가능

Labels는 왜 env로 못 쓰는가?

특성	환경변수	Volume 파일
변경 가능성	❌ 프로세스 시작 시 고정	✅ 파일 갱신 가능
Labels/Annotations	런타임에 변경 가능	—
결론	변경 불가 → 부적합	자동 갱신 → 적합

7장 ConfigMap Volume의 심볼릭 링크 원자적 교체와 동일한 메커니즘

Downward API를 쓰는 시점

상황	방법
기존 앱이 특정 env를 기대	env 방식
라벨/어노테이션 런타임 읽기	Volume 방식
Pod 이름, IP 로깅	env 방식
다른 리소스 필요	→ API Server

Part 2: API Server 접근

Downward API의 한계

Pod 자기 자신의 정보만 제공

다른 Pod, Service, ConfigMap 조회 → API Server 직접 호출

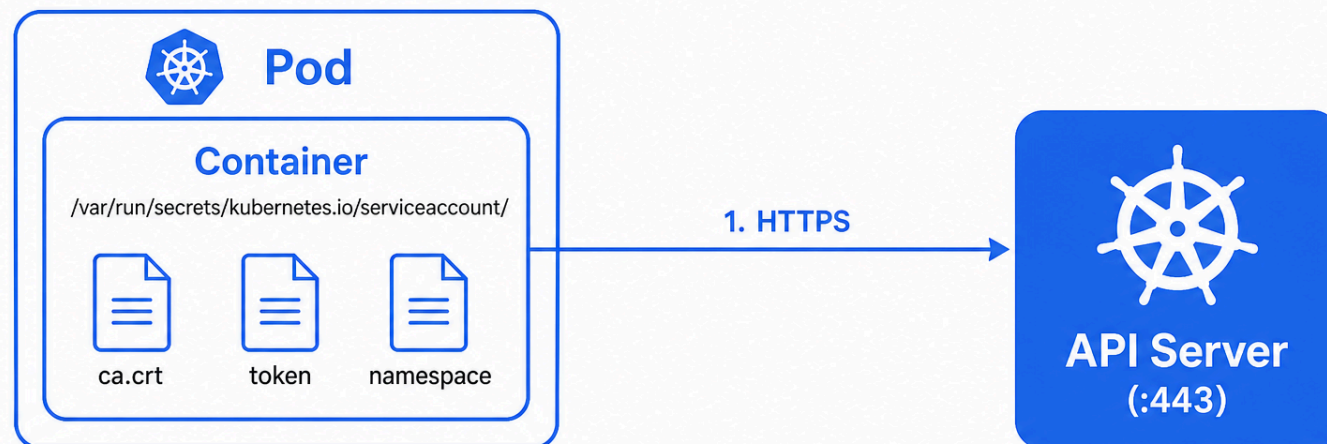
3가지 필수 요소

```
/var/run/secrets/kubernetes.io/serviceaccount/  
├── ca.crt      → 서버 인증서 검증 (MITM 방지)  
├── token       → Bearer 인증 (나는 누구인가)  
└── namespace  → 현재 Namespace
```

모든 Pod에 자동 마운트 (7장의 default-token Secret)

API Server 접근 흐름

API Server Access from Pod



- 1 **Step 1:** Find address: `https://kubernetes` ➔ **DNS** 
- 2 **Step 2:** `--cacert ca.crt` → Verify server cert ✓ ➔
- 3 **Step 3:** `-H Authorization: Bearer $TOKEN` → Client auth ✓ ➔
- 4 **Step 4:** `/api/v1/namespaces/$NS/pods` → Response ←

데모: API Server 인증 실패/성공

```
# 1. 인증 없이 → 거부
$ curl -k https://kubernetes/api
"User \"system:anonymous\" cannot get path \"/api\" → 403

# 2. CA + Token으로 인증 → 성공!
$ TOKEN=$(cat ../token)
$ curl --cacert ../ca.crt \
    -H "Authorization: Bearer $TOKEN" \
    https://kubernetes/api
{
  "kind": "APIVersions",
  "versions": ["v1"]
}
```


데모: Pod 목록 조회

```
# Namespace 파일에서 읽기
$ NS=$(cat ../namespace) # → "default"

# 같은 Namespace의 Pod 조회
$ curl -H "Authorization: Bearer $TOKEN" \
    https://kubernetes/api/v1/namespaces/$NS/pods

→ curl, downward, downward-volume
```

RBAC 주의: default ServiceAccount는 기본 권한 없음

→ 12장에서 적절한 Role/RoleBinding 구성 예정

API 경로 구조

```
Core API (apiVersion: v1):  
  /api/v1/namespaces/{ns}/pods/{name}  
  /api/v1/namespaces/{ns}/services/{name}
```

```
Named Group (apiVersion: batch/v1):  
  /apis/batch/v1/namespaces/{ns}/jobs/{name}
```

```
Non-namespaced:  
  /api/v1/nodes/{name}  
  /api/v1/persistentvolumes/{name}
```

```
kubectl get job my-job -o json ≡ GET /apis/batch/v1/namespaces/default/jobs/my-job
```

Part 3: Ambassador 패턴

문제

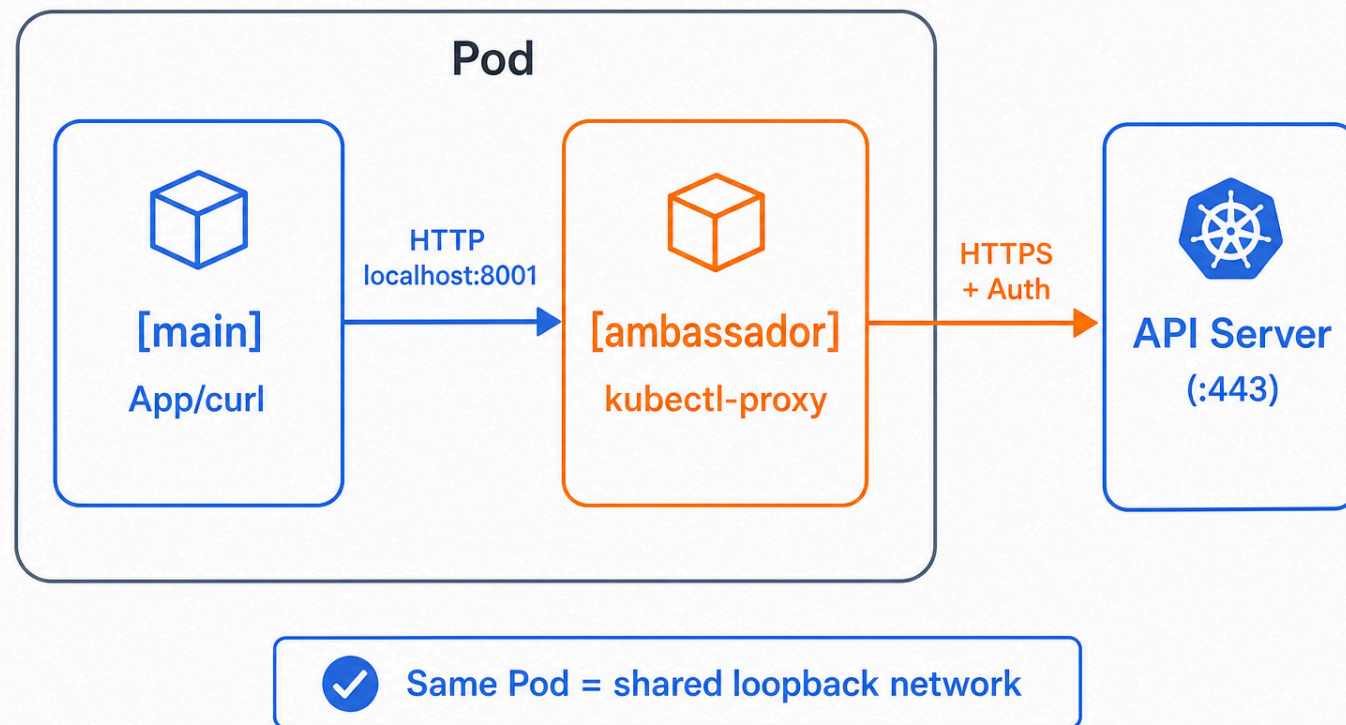
앱에서 직접 처리해야 할 것들:

- x HTTPS (TLS)
- x CA 인증서 검증
- x Bearer Token 관리
- x 인증 에러 처리

해결: kubectl-proxy를 Sidecar로 → 앱은 `localhost:8001` 에 HTTP만 보내면 됨

Ambassador 구조

Ambassador Container Pattern



데모: Ambassador 실전

```
spec:
  containers:
    - name: main
      image: curlimages/curl:8.7.1
      command: ["sleep", "9999999"]
    - name: ambassador
      image: luksa/kubectl-proxy:1.6.2
```

```
# main 컨테이너에서 - 토큰 없이, HTTP로!
$ curl localhost:8001/api
{"kind": "APIVersions", "versions": ["v1"]}

$ curl localhost:8001/api/v1/namespaces/default/pods
→ 정상 응답 (ambassador가 인증 대리)
```

Ambassador 장단점

장점	단점
앱에 K8S 인증 로직 불필요	추가 프로세스 = 메모리 소비
모든 언어에서 HTTP 호출 가능	Pod YAML 복잡성 증가
Ambassador 이미지 재사용 가능	
기존 앱 수정 없이 적용	

대안: K8S 클라이언트 라이브러리

- Go: `client-go` (공식)
- Python: `kubernetes-client` (공식)
- 자동으로 ServiceAccount 토큰 읽어 인증

의사결정 트리

Q: 어떤 정보가 필요한가?

- Pod 자기 자신 (이름, IP, 라벨, 리소스)
 - └ Downward API
 - └ 변경 가능? → Volume
 - └ 고정값? → env (더 간단)
- 다른 리소스 (Pod, Service, ...)
 - └ API Server
 - └ 앱 코드 수정 가능 → 클라이언트 라이브러리
 - └ 수정 불가 → Ambassador 패턴

default-token Secret 총정리

```
/var/run/secrets/kubernetes.io/serviceaccount/  
├── ca.crt      → 8장: API Server HTTPS 검증  
├── token       → 8장: Bearer 인증  
└── namespace   → 8장: API 경로 구성
```

7장에서 "모든 Pod에 자동 마운트되는 Secret"으로 소개됨

→ 8장에서 실제 사용처가 밝혀짐

핵심 요약

Downward API: Pod 자신의 메타데이터를 env/Volume으로

API Server: 다른 리소스는 HTTPS + Bearer Token으로 직접 조회

Ambassador: 인증 복잡성을 Sidecar가 대신 처리

Q&A

예상 질문

질문	답변
Downward API vs ConfigMap?	ConfigMap: 사용자 정의 설정 Downward: K8S 런타임 결정값
Ambassador vs Init Container?	Init: 한 번만 실행 Ambassador: 계속 프록시
ServiceAccount 토큰 보안?	RBAC 최소 권한 + Bound Token (v1.22+)